

# Module 1 — Analysis of Algorithms (Chapter 1)

Reading: book §1.1–§1.6, pp. 3–30. Coursera: "Analysis of Algorithms" lecture 1. Goal of this module: understand *what* precise algorithm analysis is (beyond Big-O), and see the whole method executed once, end to end, on quicksort.

## 1. The big idea: exact expected cost (not just Big-O)

The crudest way to summarize how long an algorithm takes is a growth rate: "about  $N \log N$  operations for an input of size  $N$ ," written  $O(N \log N)$ . (If Big-O is fuzzy, Module 0 §0.6 pins it down — for now, read  $O(N \log N)$  as "grows proportionally to  $N \log N$  for large  $N$ .") That captures the *shape* of the growth but deliberately throws away two things we actually care about: the constant factor and the lower-order terms. Those are exactly what decide whether one  $O(N \log N)$  algorithm beats another in practice.

This book's program (Sedgewick & Flajolet) is more ambitious. For a given algorithm we want:

- an **exact** formula for the average cost (as a function of the input size  $N$ ), and
- a clean **asymptotic** approximation of that formula (leading term *and* constant),
- that we can **validate empirically** by actually running the code and counting.

The payoff statement for quicksort will not be " $O(N \log N)$ ." It will be

$$\text{average number of compares} = 2(N+1) \left( H_{N+1} - \frac{3}{2} \right) \approx 2N \ln N - 1.846 N$$

— an exact expression and a precise approximation. That is a different sport from Big-O.

### Worst case vs. average case

Two ways to summarize cost over all inputs of size  $N$ :

- **Worst case** — the maximum cost over all inputs. Safe, pessimistic, often easy. This is what "complexity theory" and most courses emphasize. Quicksort's worst case is  $\sim N^2/2$  compares (already-sorted input, with this partitioning).
- **Average case** — the *expected* cost, averaging over all inputs under some probability model (here: all  $N!$  orderings of distinct keys equally likely).

Harder, but usually far closer to what you observe, and it's the focus of this book.

Average-case analysis is worth the extra math because it lets us:

- compare different algorithms for the same task realistically,
- predict actual time/space for an application,
- compare different machines running the same algorithm,
- **tune parameters** (cutoffs, sample sizes) to optimize performance.

The last one is the killer app: you can't optimize a parameter you can only bound loosely. You need the actual cost as a function of the parameter.

---

## 2. The cost model: count characteristic operations

---

We don't count nanoseconds. Machine details (caching, pipelines) are messy and machine-specific. Instead:

**Step 1 — pick a few "characteristic" operations** the algorithm's cost is built from. For quicksort the natural three are:

$A$  = number of partitioning stages,    $B$  = number of exchanges,    $C$  = number of compares

**Step 2 — total time is a linear combination** of their frequencies. On a typical machine quicksort's running time looks like

$$4C + 11B + 35A.$$

The coefficients (4, 11, 35) are machine/compiler specific; the frequencies  $A, B, C$  are properties of *the algorithm on the input*. So we analyze  $A, B, C$  once — mathematically, machine-independently — and plug in machine constants later only if we want wall-clock predictions. This separation is the whole trick that makes the analysis reusable.

Aside — why quicksort beats mergesort in practice even though both are  $\sim N \log N$ : the *cost per compare* (the coefficient on  $C$ ) is lower for quicksort. Big-O can't see that; this framework can.

Usually only a few "true" frequency variables matter, and everything else is expressible in terms of them (e.g. for quicksort,  $B$  and  $A$  both reduce to  $C$  — see the problem set). So the crux of the entire analysis is: **find  $C$ , the average number of compares.**

---

### 3. Quicksort, precisely (the worked example)

---

#### The algorithm

Partition the array around a *partitioning element* (here, the last element): scan a left pointer right until it hits something  $\geq$  the pivot, scan a right pointer left until it hits something  $\leq$  the pivot, exchange them, repeat until the pointers cross; then put the pivot in place. Now everything left of the pivot is smaller, everything right is larger. Recurse on the two sides. This implementation stops on subarrays of size  $\leq 1$ .

Model assumption: the array holds  $N$  randomly ordered, **distinct** numbers — i.e. each of the  $N!$  orderings is equally likely. (The code is correct for any input; this is just the probability model we analyze under.)

#### Setting up the recurrence for $C_N$

Let  $C_N$  = average number of compares to sort  $N$  elements. Base cases:  $C_0 = C_1 = 0$  (nothing to compare in an array of size 0 or 1).

For  $N > 1$ , one partitioning stage does  $N + 1$  compares (each of the  $N$  elements is compared to the pivot, plus one extra where the pointers cross). After partitioning, the pivot lands in some final position. Here's the key probabilistic step:

**The pivot is equally likely to end up in any of the  $N$  positions.** Concretely, the pivot is the  $j$ -th smallest element with probability  $\frac{1}{N}$ , for each  $j = 1, 2, \dots, N$ . When it's the  $j$ -th smallest, the left subarray has size  $j - 1$  and the right has size  $N - j$ .

This is the one combinatorial fact the whole analysis rests on. It holds because a random permutation, after partitioning, leaves two subarrays that are *themselves* randomly ordered ("randomness preservation" — see §5 below) and the pivot's rank is uniform.

So, averaging the cost of the two recursive calls over all  $N$  equally-likely pivot ranks:

$$C_N = (N + 1) + \frac{1}{N} \sum_{1 \leq j \leq N} (C_{j-1} + C_{N-j}). \quad (\star)$$

Read that as: cost of this stage ( $N + 1$ ), plus the average over  $j$  of [cost of left subarray of size  $j - 1$ ] + [cost of right subarray of size  $N - j$ ].

**This is the moment the analysis leaves computer science behind.** Equation  $(\star)$  is a pure math problem — no code, no machine, just a recurrence. Solving it is the rest of the job. This "derive a recurrence that mirrors the recursion, then solve it" pattern is how the whole book analyzes recursive algorithms.

## Solving the recurrence

The sum  $\sum (C_{j-1} + C_{N-j})$  runs over the *same* set of values twice, just in opposite order: as  $j$  goes  $1, \dots, N$ , the term  $C_{j-1}$  gives  $C_0, \dots, C_{N-1}$ , and  $C_{N-j}$  gives  $C_{N-1}, \dots, C_0$ . Same values. So the two halves are equal, and

$$C_N = N + 1 + \frac{2}{N} \sum_{1 \leq j \leq N} C_{j-1} \quad \text{for } N > 0.$$

Now the standard move to kill the sum: **multiply by  $N$ , then subtract the equation for  $N - 1$** .

$$\begin{aligned} N C_N &= N(N + 1) + 2 \sum_{1 \leq j \leq N} C_{j-1} \\ (N - 1) C_{N-1} &= (N - 1)N + 2 \sum_{1 \leq j \leq N-1} C_{j-1} \end{aligned}$$

Subtract; the sums collapse to a single leftover term  $2 C_{N-1}$ :

$$N C_N - (N - 1) C_{N-1} = N(N + 1) - (N - 1)N + 2 C_{N-1} = 2N + 2 C_{N-1}.$$

Rearrange:

$$N C_N = (N + 1) C_{N-1} + 2N \quad \text{for } N > 1.$$

Now the elegant step: **divide by  $N(N + 1)$**  so the recurrence telescopes. This turns  $C_N/(N + 1)$  into "previous term + something summable":

$$\frac{C_N}{N + 1} = \frac{C_{N-1}}{N} + \frac{2}{N + 1}.$$

Let  $T_N = C_N/(N + 1)$ . Then  $T_N = T_{N-1} + \frac{2}{N+1}$ , so iterating from  $T_1 = C_1/2 = 0$ :

$$\frac{C_N}{N + 1} = \frac{C_1}{2} + 2 \sum_{3 \leq k \leq N+1} \frac{1}{k}.$$

The sum of  $\frac{1}{k}$  is a **harmonic number** (see §4). Since  $\sum_{3 \leq k \leq N+1} \frac{1}{k} = H_{N+1} - 1 - \frac{1}{2}$ ,

$$C_N = (N + 1) \cdot 2 \left( H_{N+1} - \frac{3}{2} \right) = 2(N + 1) \left( H_{N+1} - \frac{3}{2} \right).$$

That's the exact average number of compares. Done.

## Reading the answer

Using  $H_{N+1} \approx \ln N$  (next section):

$$C_N \approx 2N \ln N - 1.846 N.$$

Converting  $\ln$  to  $\lg$  (since  $2 \ln N = 2(\ln 2) \lg N \approx 1.39 \lg N$ ):

$$C_N \approx 1.39 N \lg N.$$

So on average quicksort uses about **39% more compares than the  $\sim N \lg N$  information-theoretic minimum** — and the book notes this analytic curve fits actual measured compare-counts essentially perfectly (Figure 1.1: gray dots = single runs, black = means, the formula threads right through them). That empirical validation is the whole point: the math *predicts* reality, not just bounds it.

---

## 4. The harmonic numbers $H_N$ (your first "special" sequence)

---

$$H_N = \sum_{1 \leq k \leq N} \frac{1}{k} = 1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{N}.$$

These show up constantly in analysis of algorithms (they're the discrete analog of  $\int dx/x$ ). The key approximation, which we'll prove properly in Module 4:

$$H_N \approx \ln N + \gamma, \quad \gamma = 0.57721 \dots \text{ (Euler's constant).}$$

Intuition for why:  $H_N = \sum \frac{1}{k}$  is a right-hand rectangle approximation to  $\int_1^N \frac{1}{x} dx = \ln N$ . The rectangles overshoot the curve by a bounded amount that converges to  $\gamma$ . So  $H_N$  grows like  $\ln N$ , slowly and forever. This single fact is *why* the  $\ln N$  appears in quicksort's cost — the harmonic sum came straight out of the telescoping recurrence.

Small values to build intuition:

$H_1 = 1$ ,  $H_2 = 1.5$ ,  $H_4 \approx 2.083$ ,  $H_{10} \approx 2.929$ ,  $H_{100} \approx 5.187$ . Compare  $\ln 100 + \gamma \approx 4.605 + 0.577 = 5.182$ . Already excellent at  $N = 100$ .

---

## 5. Two caveats worth internalizing

---

**Randomness preservation.** The clean recurrence ( $\star$ ) only works because *after* partitioning a random permutation, the two subarrays are themselves random permutations. Not every quicksort variant preserves this! (E.g. initializing the right pointer wrong breaks it — book Exercise 1.13.) Variants that *don't* preserve randomness are both harder to analyze *and*, empirically, tend to perform worse — a nice case where "analyzable" and "good" line up. When you design an algorithm, preserving a clean probabilistic invariant is worth something.

**Exact vs. asymptotic.** We got an *exact* answer here,  $2(N+1)(H_{N+1} - \frac{3}{2})$ . That's the ideal. But often exact answers are unavailable or unreadable, and a concise

approximation ( $2N \ln N - 1.846N$ ) is both easier to get and easier to interpret. The book's stance: aim for exact, settle gracefully for precise-asymptotic. Chapter 4 is the machinery for the "settle gracefully" part. Harder variants (median-of-three quicksort, radix-exchange sort) give recurrences too hard for the elementary trick above — those need generating functions (Ch 3) and asymptotics (Ch 4). That's the motivation for the whole rest of Part I.

---

## 6. What to take away

---

1. **Analysis of algorithms  $\neq$  Big-O.** We compute exact average cost + a precise asymptotic, and check it against experiment.
2. **Cost = linear combination of characteristic-operation frequencies.** Analyze the frequencies ( $A, B, C$ ) machine-independently; multiply by machine constants only at the end.
3. **Recursive algorithm  $\rightarrow$  recurrence  $\rightarrow$  solve it.** The recurrence *mirrors* the recursion. Setting it up needs a probability model (here: uniform pivot rank,  $\frac{1}{N}$ ).
4. **Standard recurrence-solving toolkit** shown once here: exploit symmetry to simplify the sum, multiply-and-subtract to kill the sum, divide by the right factor to telescope, recognize the resulting sum (harmonic numbers).
5.  $H_N \approx \ln N + \gamma$  is the reason "log" appears — remember it.
6. The hard variants motivate Chapters 2–4. Onward.

$\rightarrow$  Now do `problem-sets/module-01-problems.md`. The derivations there are where this actually sinks in — reproduce the quicksort solve from scratch without looking.